Task 1: Two-Factor Authentication System

Task 1a: Biometric Authentication

Introduction

This report aims to determine the choice of threshold on the security and usability of the authentication system with the help of the calculated results of False Acceptance Rate (FAR) and False Rejection Rate (FRR) with the two threshold settings 0.95 and 0.97.

Testing Results

The two tables below contain the results of two people using a face recognition software to login into the same account 10 times. Table 1 contains the results of "Ammie", a registered person; Table 2 contains the results of "Brenda", an un-registered person trying to login to Ammie's account.

Table 1:

Registered Person ("Ammie") Testing Image ID	Similarity Probability Score
1	0.985
2	0.971
3	0.984
4	0.977
5	0.996
6	0.999
7	0.968
8	0.956
9	0.935
10	0.990

Table 2:

Un-Registered Person ("Brenda") Testing Image ID	Similarity Probability Score
1	0.923
2	0.922
3	0.930
4	0.940
5	0.955
6	0.937
7	0.963
8	0.909
9	0.982
10	0.943

Login Results

With the different threshold settings of 0.95 and 0.97, below are the results of Ammie and Brenda trying to login to Ammie's account.

0.95:

- Ammie True Positive (TP): 9
- Ammie False Negative (FN): 1
- Brenda False Positive (FP): 3
- Brenda True Negative (TN): 7

0.97:

- Ammie True Positive (TP): 7
- Ammie False Negative (FN): 3
- Brenda False Positive (FP): 1
- Brenda True Negative (TN): 9

Computation of FAR and FRR Metrics

Based on the results obtained above, the FAR and FRR metrics using the two threshold settings of 0.95 and 0.97 is computed. (A = Ammie's total attempts, B = Brenda's total attempts)

0.95:

- Brenda FAR: FP/B = 3/10 = 0.3 (30% impersonation rate)

- Ammie FRR: FN/A = 1/10 = 0.1 (10% self-rejection rate)

0.97:

- Brenda FAR: FP/B = 1/10 = 0.1 (10% impersonation rate)

- Ammie FRR: FN/A = 3/10 = 0.3 (30% self-rejection rate)

Security and Usability

Using the results obtained above, the security and usability of the two threshold settings can be determined.

0.95:

With a 30% impersonation rate, this setting is incredibly insecure as an unauthorized user can login to another user's account quite easily. However, with a 10% self-rejection rate, this setting is tolerable and usable for most users.

0.97:

With a 10% impersonation rate, this setting is much more secure as there is a much lower chance of an unauthorized user logging into another user's account. However, with a 30% self-rejection rate, a lot of users may find this setting to be intolerable, drastically worsen their experience.

Conclusion

In conclusion, after computing the FAR and FRR with the two threshold settings 0.95 and 0.97, these settings are give-and-take. This means that a lower threshold setting (0.95) will provide better usability but worse security while a higher threshold setting (0.97) will provide better security but worse usability.

Task 1b: Password Authentication

Introduction

This report aims to hack four system passwords of different hashing methods using John the Ripper (command john) and investigate the time it takes to brute force these passwords as well as the time to compute these same passwords before determining which hashing method should be used based on usability and security considerations.

Brute Forcing

Using John the Ripper's default dictionary, the passwords are brute forced. The time taken to do so for the different passwords can be seen below.

SHA-512 5000 rounds (Default):

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 2/1b$ time john SHA-512_Default.hash
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
world            (?)
1g 0:00:00:04 100% 2/3 0.2040g/s 666.1p/s 666.1c/s 666.1C/s spazz..dasha
Use the "--show" option to display all of the cracked passwords reliably
Session completed

real    0m4.919s
user    0m4.726s
sys     0m0.065s
```

SHA-512 1000 rounds:

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 2/1b$ time john SHA-512_1000_rounds.hash
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
world            (?)
1g 0:00:00:01 100% 2/3 0.9615g/s 3138p/s 3138c/s 3138C/s spazz..dasha
Use the "--show" option to display all of the cracked passwords reliably
Session completed

real    0m1.049s
user    0m1.005s
sys     0m0.023s
```

SHA-512 10000 rounds:

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 2/1b$ time john SHA-512_10000_rounds.hash
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
world            (?)
1g 0:00:00:09 100% 2/3 0.1019g/s 332.7p/s 332.7c/s 332.7C/s spazz..dasha
Use the "--show" option to display all of the cracked passwords reliably
Session completed

real    0m9.820s
user    0m9.561s
sys     0m0.053s
```

SHA-256 5000 rounds (Default):

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 2/1b$ time john SHA-256_Default.hash
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
world            (?)
1g 0:00:00:06 100% 2/3 0.1526g/s 498.3p/s 498.3c/s 498.3C/s spazz..dasha
Use the "--show" option to display all of the cracked passwords reliably
Session completed

real    0m6.548s
user    0m6.379s
sys     0m0.089s
```

The difference in these methods is that the first three use SHA-512 for a different number of rounds, the rounds being 5000, 1000 and 10000. The fourth method uses SHA-256 (Default 5000 rounds), a different algorithm from the first three.

As observed from the results above, each method takes a different amount of time to brute force.

Dictionary of 500 Million Passwords

Using the command below, it can be observed that John the Ripper has a dictionary of 3559 passwords.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 2/1b$ wc -l /usr/share/john/password.lst
3559 /usr/share/john/password.lst
```

What if he had a dictionary of 500 million passwords? How long would it take to brute force each method?

The time taken can be estimated by using the method below.

500,000,000 / 3559 * time taken for 3559 = 140489 * time taken for 3559

SHA-512 5000 rounds (Default):

140489 * 4.726 = 663951.014s = 184.431hr

SHA-512 1000 rounds:

140489 * 1.005 = 141191.445s = 39.220hr

SHA-512 10000 rounds:

140489 * 9.561 = 1343215.329s = 373.115hr

SHA-256 5000 rounds (Default):

140489 * 6.379 = 896179.331s = 248.939hr

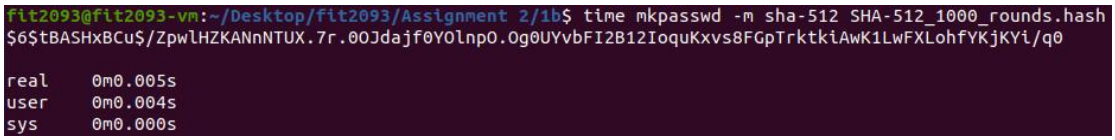
Single Password Hash

The results for the time taken to compute a single password hash by SHA-512 which is the time taken for a server to verify one login can be seen below.

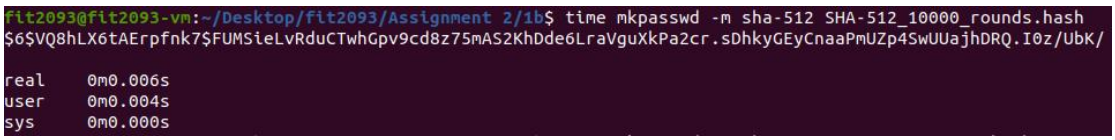
SHA-512 5000 rounds (Default):



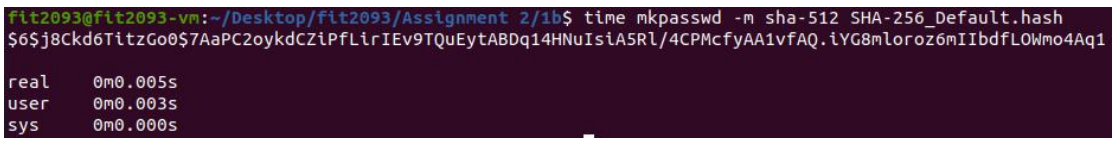
SHA-512 1000 rounds:



SHA-512 10000 rounds:



SHA-256 5000 rounds (Default):



As observed from the results above, every method takes roughly the same amount of time.

Conclusion and Recommendations

After evaluating the time taken to brute force as well as verify the login for all four methods, I have come to the conclusion that SHA-512 (10000 rounds) is the best method for password hashing.

This is because SHA-512 has a bigger hash size than SHA-256. Furthermore, there is no difference in time taken for the system to verify the login for all methods regardless of hash size or rounds of operation. Therefore, not only is SHA-512 (10000 rounds) secure thanks to its large hash size and large number of rounds, but it is also usable as it takes almost no time at all to verify the login.